

Programación III — Material Teórico-Práctico

Tema: Resolución de Recurrencias en Algoritmos Recursivos

Cátedra: Programación III

Métodos: Sustracción y División

 **Resumen del Apunte:** Este documento sirve como guía teórica complementaria para el análisis de complejidad temporal en funciones recursivas. Explica cómo plantear la ecuación de costo $T(n)$ y resolverla aplicando los métodos de **Sustracción** y **División**.

1. Introducción a las Ecuaciones de Recurrencia

A diferencia de los algoritmos iterativos donde contamos ejecuciones de bucles, en los algoritmos recursivos el costo de resolver un problema de tamaño n se expresa en función del costo de resolver subproblemas de menor tamaño. Esta relación matemática se denomina **Ecuación de Recurrencia**.

Ejemplo Básico: Función Factorial

```
int factorial(int n) {  
    if (n == 1) {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}
```

La función de costo $T(n)$ para este algoritmo se define como:

$$T(n) = c \text{ si } n = 1 \mid T(n) = T(n - 1) + c \text{ si } n > 1$$

2. Método de Sustracción: $T(n - b)$

Se aplica cuando en cada llamada recursiva el tamaño de la entrada disminuye en una **cantidad constante de unidades b** .

Estructura General:

$$T(n) = c \quad \text{si } 0 \leq n < b$$
$$T(n) = a \cdot T(n - b) + p(n) \quad \text{si } n \geq b$$

- **a**: Cantidad de llamadas recursivas realizadas en el peor caso.
- **b**: Unidades en que disminuye la entrada en cada llamado.
- **k**: Grado del polinomio $p(n)$ correspondiente al trabajo realizado fuera de la llamada recursiva.

Regla de Clasificación Asintótica (Sustracción):

Condición de 'a'	Complejidad Temporal Resultante
$a < 1$	$\Theta(n^k)$
$a = 1$	$\Theta(n^{k+1})$
$a > 1$	$\Theta(a^{n/b})$

 **Caso de Estudio (Factorial):** Para `factorial(n)` tenemos $a = 1$, $b = 1$ y $k = 0$. Como $a = 1$, aplicamos $\Theta(n^{k+1}) = \Theta(n^{0+1}) = \Theta(n)$ (Complejidad Lineal).

3. Método de División: $T(n / b)$

Se aplica cuando en cada llamada recursiva el tamaño de la entrada se **divide por un factor constante b** (estrategia *Divide y Vencerás*).

Estructura General:

$$T(n) = c \quad \text{si } 0 \leq n < b$$
$$T(n) = a \cdot T(n / b) + f(n) \quad \text{si } n \geq b$$

- **a**: Cantidad de llamadas recursivas realizadas en el peor caso.
- **b**: Factor por el cual se divide el tamaño de la entrada.
- **k**: Grado del polinomio $f(n)$ del trabajo realizado fuera de la llamada recursiva.

Regla de Clasificación Asintótica (División):

Condición Comparativa	Complejidad Temporal Resultante
$a < b^k$	$\Theta(n^k)$
$a = b^k$	$\Theta(n^k \cdot \log(n))$
$a > b^k$	$\Theta(n^{\log_b(a)})$

Ejemplo Práctico: Potenciación Rápida $\text{exp2}(a, b)$

```
public long exp2(long a, long b) {  
    if (b == 0) {  
        return 1;  
    } else {  
        long result = exp2(a * a, b / 2);  
        if (b % 2 == 1) {  
            result = a * result;  
        }  
        return result;  
    }  
}
```

Análisis de Parámetros:

- Llamada recursiva dividida por 2: $b = 2$
- Número de llamadas por ejecución: $a = 1$
- Trabajo fuera del llamado: Operaciones elementales $O(1) \Rightarrow k = 0$

Evaluación: Calculamos $b^k = 2^0 = 1$. Como $a = 1$ y $b^k = 1$, se cumple la condición $a = b^k$. Por ende, la complejidad es $\Theta(n^0 \cdot \log(n)) = \Theta(\log n)$.

4. Cuadro Comparativo de Referencia Rápida

Método	Reducción Entrada	Criterio Clave	Ejemplo Clásico
Sustracción	$n - b$	Comparar a con respecto a 1	Factorial, Torres de Hanói
División	n / b	Comparar a con respecto a b^k	Búsqueda Binaria, MergeSort, Potencia Rápida